CS4225/5425 Big Data Systems, Data Science

AY24/25 Sem 2. github.com/gerteck

(Big) Data science. Interdisciplinary field about processes and systems to extract knowledge or insights from data in various forms.

- **Processing** diff. types of data (large data volume, graph data, stream data).
- **Models of Computation**: MapReduce/Spark, Large graph processing engines, Streams and online algorithms.

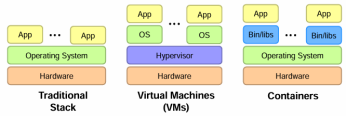
1. Introduction

- **Big Data (4 'V's)**: *Volume*: (Scale of data). *Velocity*: (Speed of new (streaming) data). *Variety*: (Different types). *Veracity*: (Accuracy, trustworthiness of data).
- **Challenges of 4 'V's**: Performance, Cost, Reliability, Algorithm design complexity. Data integration challenges, multi-modal learning (Variety). Dirty and noisy data, data provenance, uncertainty (Veracity).
- Big data - Virtuous Product Cycle, where useful service → data to analyze → insightful action → useful service.

Utility Computing (Cloud Computing Infrastructure)

Cloud computing - scalable, elastic infrastructure for big data systems.

- **Utility Computing**: Computing resources as a metered service ("pay as you go"). Ability to dynamically provision virtual machines.
- **Scalable**: ("Infinite" capacity). **Elastic**: (Scale up or down on demand).



- **How**: **Virtual Machines (VMs)**: Enable sharing of hardware resources, run each app in isolated virtual machine. High overhead as each VM has its own OS. **Containers**: Lightweight sharing of resources. Apps run in isolation but share same OS. Container encapsulates an app and environment.
- **Everything as a Service**:
  - *Infrastructure (IaaS)*: User rents a virtual machine. (e.g. Amazon EC2).
  - *Platform (PaaS)*: Provide hosting web apps. Cover hardware maintain, upgrade.
  - *Software (SaaS)*: Pre-built apps. (e.g., Gmail). Covers entire stack.

Data science, cross-disciplinary, emerging research area w. applications in engineering, commerce, etc. Clouds as natural infrastructures for data science.

2. Principles of Big Data Systems

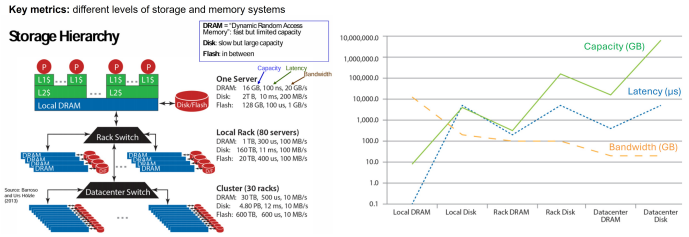
- **Data Center Infrastructure**
  - **Single Server**: A commodity hardware unit.
  - **Rack**: Dozens of servers, connect to rack switch (top-of-rack switch).
  - **Data Center**: Hundreds of racks, connect to data center switch (core switch).
- **Bandwidth vs. Latency**
  - **Bandwidth**: Max amt of data transmittable per unit time (e.g., in GB/s). Calculate as minimum bandwidth among all components in the data access path.
  - **Latency**: Time taken one packet travel src to dest (one-way) / round-trip (in ms). Sum of the latency caused by all components in the data access path.
  - **Throughput**: Rate data is actually transmitted in network in given time period.
- When transmitting *(large/small)* amount of data, *(bandwidth/latency)* - roughly how long transmission will take (bottleneck variable).
- **Storage Hierarchy in Data Centers**: based on capacity, latency, bandwidth.
  - **DRAM**: Fast but limited capacity. **Flash**: Intermediate btw. DRAM and disk.
  - **Disk**: Slow but large capacity. Disk reads more expensive than DRAM (higher latency, lower bandwidth).

Data Flow Paths

Data flow paths depend on the location of the source and destination. Data access may cross several components, e.g. hard disk, DRAM, rack switch, cluster switch.

- E.g. Local DRAM → Local Disk in same server.
- Server A DRAM to server B DRAM in same rack (i.e. Local DRAM → Rack Switch → Server B's DRAM)
- Transfer across different rack, same cluster etc. (i.e. Local DRAM → Rack Switch → Datacenter Switch → Server C's Rack Switch → Server C's DRAM).
- To load from disk, usually need to load into DRAM first. (i.e. Local Disk → Local DRAM → Rack Switch etc.)

Calculate latency, bandwidth based on data flow path. Approximate order of magnitude, for simplify, assume one-way communication, no failures.



Capacity increases, **Costs** (Latency increase, bandwidth decrease) increases from local to rack to data center, along storage hierarchy.

Building Efficient Big Data Systems

1. **Scale "Out", Not "Up"**: Horizontal Scaling over vertical scaling. Combine many cheaper machines (out) over increase power of individual machine (up).
2. **Seamless Scalability**: Ideal (linear) scalability, performance increases proportionally to added resources.
3. **Move Processing to the Data**: As clusters limited bandwidth; move tasks to machine where data is stored.
4. **Process Data Sequentially, Avoid Random Access**: Disk seeks expensive; sequential access provides better throughput.

Challenges in Big Data Systems

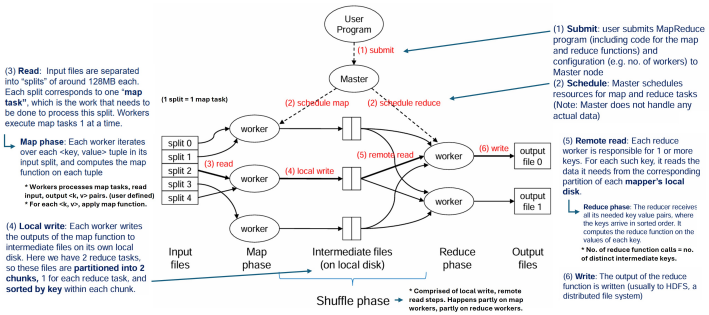
1. **Machine Failures**: With enough scale, expect daily server machine failure.
2. **Synchronization**: Difficult due to unknown order of worker execution, interruptions, and shared data access. Tools like semaphores, conditional variables, and barriers are used to address synchronization issues.
3. **Programming Difficulty**: Concurrency hard to reason about, especially at scale. Debugging and managing failures add complexity.

API for Data Centers: Given challenges, "API" of data center abstracts system-level details from developers. Execution frameworks (e.g., Hadoop, Spark) handle task assignment, replication, synchronization, etc.

3. MapReduce

Big data requires new prog. abstractions, runtime systems. Hadoop MapReduce widely used for efficient, process large datasets in e.g. databases, data mining. **MapReduce**: prog. model & associated runtime system to process large datasets in parallel across distributed cluster. (Parallel + Divide & Conquer Paradigm). MapReduce provides functional abstractions for two operations.

- **Map function**: `(k1, v1) → List(k2, v2)`. Processes each input key-value pair and emits intermediate key-value pairs.
- **Reduce**: Aggregate intermediate results to generate the final output. `reduce(k2, List(v2)) → List(k3, v3)`. All values with same key sent to same reducer.
- **Typical Workflow of MapReduce**:
  Iterate over a large number of records. Extract relevant info from each. (**Map**)
  Shuffle and sort intermediate results by key (**Shuffle**).
  Aggregate intermediate results (**Reduce**). Generate the final output.



- **Worker**: single component of cluster, performs storage, processing tasks.
- **Map Task**: A unit of work, processing one input split (e.g., 128 MB).
- **Map Function**: A call to user-defined map function, for each input split k-v pair.
- **Reduce Task**: A unit of work processing a subset of intermediate keys.
- **Barrier**: barrier between map and reduce phases. Else, reduce phase may compute wrong answer. Shuffle phase can begin copying intermediate data earlier.

Partition and Combiner

Optionally specify **partition, combine** functions, optimize, reduce network traffic.

- **Partition Step**: By default, keys assigned to reducers using hash function (e.g., `hash(key) % num_reducers`). Can implement custom partitioner to balance load among reducers. (E.g. if some keys much more values than others).
- Partitioner runs in local write phase, knowledge of which keys to which reducer.
- **Combiner Step**: Combiners locally aggregate map outputs to reduce disk and network I/O. ('mini-reducers') (E.g. Sum counts for each key before write to disk). *Correctness*: Must not affect correctness of final output, regardless of how many times it runs (incld. 0 times). E.g. `sum`, `max` ok, but `mean`, `minus` not ok.
- Combiner runs after map phase, done during local write phase, before actual disk write to save disk I/O.

Notes and Other Optimizations

- If reduce task handles multiple keys, will process keys in sorted order.
- **Secondary Sort**: Use this property to sort values associated with key. (reduce work done in reducer by using inbuild distributed sorting process of Hadoop). Combine key and value into composite key e.g. (UserID → [UserID-Timestamp]), define custom comparator (sort by ID then timestamp) & partitioner (group by ID only).
- Size of split too big or small: → Limited parallelism vs. high overhead.
- #map fn calls = # input key-value pairs. # map tasks = # input splits.
- Reduce function calls = no. of distinct intermediate keys.
- **Preserving State in Map / Reduce Tasks**: In MapReduce, state variables can be used to store information that persists across multiple calls to map() or reduce() functions within same task. Sharing data or resources between function calls.

General Performance Guidelines (Basic Algorithm Design)

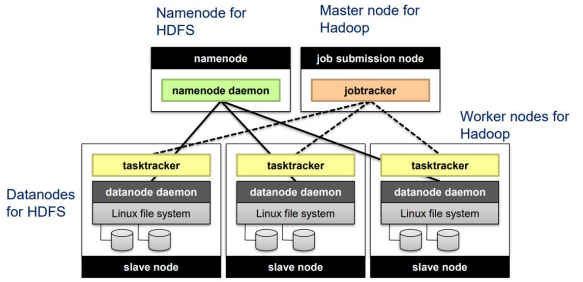
- **Linear Scalability**: More nodes should enable more work done in same time.
- **Minimize Disk and Network I/O**: Sequential access over random access. Send data in bulk over small chunks.
- **Reduce Memory Working Set**: Avoid large in-memory data structures to prevent out-of-memory errors.

4. MapReduce & Databases

HDFS: Hadoop Distributed File System

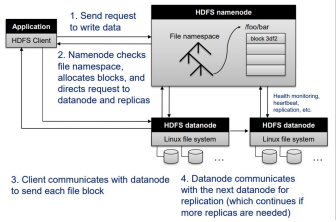
Distributed file system designed to store, manage large datasets across multiple machines in a cluster. Core component of Apache Hadoop ecosystem (open source), optimized for high-throughput access to big data.

- **Purpose**: handle massive datasets (tb/pb), scalable, fault-tolerant storage.
- E.g. MapReduce input, output stored in HDFS (3 copies etc.) Intermediate files on hard disk on local machine though (local fs).
- **Architecture**: *Master-Slave Model*.
  NameNode: Master node, manage file system metadata (e.g., dir structure, file locations, rebalance, heartbeat) - No data moved through NN. If NN data lost, cannot reconstruct files from raw block data. Hadoop has NN backup, checkpointing.
  DataNodes: Worker nodes, store the actual data blocks.

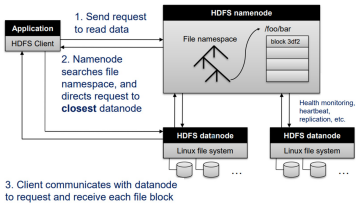


- **Data Locality**: Move computation to data, over other way round. Minimize network traffic, improve performance. Same set of nodes used for both HDFS, Hadoop. Hadoop tries to schedule map tasks run on machines containing data.
- **Fault Tolerance**: Each block replicated (default: 3 copy) across different DataNodes to ensure reliability. If node fails, HDFS retrieves data from another replica.
- **Rack Awareness**: Aware of physical location of nodes (e.g., which rack). Replicas placed on diff rack, balance fault tolerance, network efficiency.
- **Block Size**: Files divide to large chunks (default: 128 or 256 MB), reduce overhead. Large block sizes improve sequential read/write perform. (↑ throughput).
- **Scalability**: Scales horizontally by adding more machines to the cluster. Supports thousands of nodes and handles large datasets efficiently.
- **Write Once, Read Many**: Files written once, can be read multiple times. Files cannot be modified after written (append-only in some cases).
- **Use Cases**: Store log files, web server data, social media data, etc. Support big data processing frameworks like MapReduce, Spark, Hive, and Pig.

HDFS Architecture: Writing Data



HDFS Architecture: Reading Data



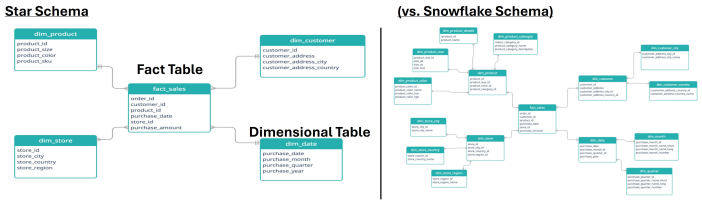
Disadvantages of HDFS: Not suited for small files, inefficient for large no. of small files, due to metadata overhead. High latency, not ideal for real-time or low-latency applications. Limited Random Access: design for seq access, not random reads/writes.

Relational Databases DB

Relational database comprised of tables, each table represents a relation, which is a collection of tuples (rows). Each tuple consists of multiple fields. Develop large-scale relational database using Hadoop / MapReduce.

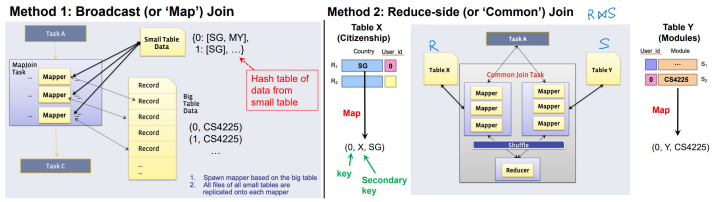
Star Schema and SQL Queries

The Star Schema is a common database schema for data warehousing. Organizes data into single (central) fact table and dimension tables. Fact table linked via foreign key r/s to PK of each dimension. Faster query and simpler queries.



SQL (Relational) Operations in MapReduce

- **Projection in SQL/MapReduce**: Projection selects specific columns from table. (E.g. `SELECT <columns> FROM Sales`). Only selected columns returned.
  – **Map Phase**: Input: Tuples (t. ID as key), emit tuples (w. only select attributes).
  – **No Reducer Needed**: "map-only" job, no need for shuffling or reducing.
- **Selection in SQL/MapReduce**: Selection filters rows based on condition. (E.g. `SELECT * FROM Sales WHERE price > 10`). Only rows satisfying the predicate (condition) are returned. MapReduce implementation:
  – **Map Phase**: Input: Tuples (tuple ID as key) emit tuples satisfying predicate.
  – **No Reducer Needed**: Similar to projection, this is a map-only job.
- **Group By and Aggregation**: Aggregation operations. Shuffle phase is the 'group by'. Framework auto groups tuples by key. (E.g. `SELECT product_id, AVG(price) FROM sales GROUP BY product_id`). Output avg per product.
  – **Map Phase**: Emit key-value pairs: ⟨product_id, price⟩.
  – **Shuffle Phase**: Framework groups tuples by key (`product_id`).
  – **Reduce Phase**: Compute average price for each `product_id`. (Same Reducer). Optimize using combiners to reduce network traffic.
- **Relational Joins (Inner Join)**: Joins combine data from two tables based on a common key. (E.g. `SELECT A.Id, A.attr1, B.attr2 FROM A JOIN B ON A.bID = B.bID`) Many-to-many r/s (can appear multiple times in both tables).
  1. **Broadcast (Map) Join**: Requires the smaller table to fit in memory. Mappers store copy of small table as hash table. Iterate over large table, perform join. (Hash Join. Smaller table 'broadcasted' to all mappers/processing nodes.)
  2. **Reduce-side (Common) Join**: Mappers emit records with join key as primary key. Reducers group records by join key. Can use secondary sorting to ensure all keys from one table arrive before the other. Hold keys from first table in memory, cross them with records from second table (to do join).



- Big data systems do not automatically guarantee performance benefits.
- Performance analysis, algorithm design crucial for efficient, effective applications.

5. MapReduce & Data Mining

Similarity Search

Similarity search is a fundamental problem in data mining and machine learning, often used to find objects "similar" to each other. Real-world applications e.g.:

- Finding documents with similar words for duplicate detection, topic classification. Clustering similar news articles, research papers.
- Identifying customers purchasing similar products for recommendation systems.
- Grouping users with similar browsing behaviors for targeted advertising.

Distance and Similarity Measures

Between objects x and y, use **distance measure** $d(x, y)$ or **similarity measure** (opposite). 'Near Neighbours' are points small distance apart. Common metrics:

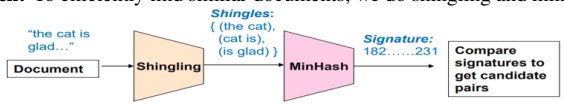
- **Euclidean Distance**: Straight-line distance btw two points in space.
- **Manhattan Distance**: Sum of absolute differences along axes.
- **Cosine Similarity**: Measures the cosine of the angle between two vectors, focusing only on direction (ignoring magnitude).
- **Jaccard Similarity**: Similarity between sets A, B: $s_{\text{Jaccard}}(A, B) = \frac{|A \cap B|}{|A \cup B|}$
- **Jaccard Distance**: Complement s_{Jaccard}: $d_{\text{Jaccard}}(A, B) = 1 - s_{\text{Jaccard}}(A, B)$

Case Study: Finding Similar Documents

Problem Definition: Given a large collection of N documents:

- **All-Pairs Similarity**: Find all pairs of docs w. Jaccard similarity above threshold.
- **Similarity Search**: Given a query document, find all documents similar to it.

Approach: To efficiently find similar documents, we do shingling and min-hash.



Shingling

- A **k-shingle** (or k-gram) is sequence of k tokens (words / chars) that appears in document. E.g. 'the cat is glad' → set of 2-shingles: {'the cat', 'cat is', 'is glad'}
- **Similarity Metric for Shingles**: Each document D_i represented as set of shingles C_i. Similarity of (D_1, D_2): $s_{Jaccard}(D_1, D_2) = \frac{|C_1 \cap C_2|}{|C_1 \cup C_2|}$
- **Challenge**: For large datasets, comparing all pairs directly computationally expensive ($O(N^2)$ comparison). To address this, use **Min-Hashing**.

Min-Hashing (Each shingle hashes to a int, get min int value of all shingles)

Convert large sets of shingles into short signatures while preserving similarity. Highly similar = high probability of same signature. Dissimilar docs low prob.

1. Define a hash function h that maps each shingle to an integer.
2. For each document, compute min hash value across all shingles → 'signature'.
3. Use *multiple* independent hash functions to generate *multiple* signatures.

- **Key Property of Min-Hashing**: probability two documents have same MinHash signature is equal to Jaccard similarity.
- **Candidate Pairs**: Compare signatures (e.g. $> k$ matching signatures). Pairs either directly used as output or further compared to confirm similarity.

MapReduce Implementation

- **Map Phase**: Read each document, extract shingles. Hash each shingle, compute MinHash signature. Emit ⟨signature, document_id⟩. (Either run multiple times for diff hash, or use signature_vector, custom partitioner to send same reducer.)
- **Reduce Phase** Receive all documents with the same MinHash signature. Generate candidate pairs from these documents. (Optional, confirm pair further similarity).
- **Parallelism**: # map node → # docs chunks. # reduce node → # keys (signature).

Clustering

- **Clustering**: Fundamental technique in data mining, machine learning to group unlabeled data points into clusters of similar items. Ideally, points have high intra-cluster similarity, low inter-cluster similarity.
- **Applications of Clustering**: Microbiology gene/protein identification, recommendation systems, social networks, information retrieval.

K-Means Algorithm

Clustering technique. Separates data points into K clusters by iteratively optimizing positions of cluster centers.

1. **Initialization**: Randomly select K points as initial cluster centers.
2. **Repeat until convergence**:
   (a) **Assignment Step**: Assign each data point to nearest cluster center based on distance metric (e.g., Euclidean distance).
   (b) **Update Step**: Move each cluster center to average (mean) position of all points assigned to it.
3. **Stopping Condition**: Stop when no assignments change between iterations (i.e., the algorithm converges).

Challenge with K-Means: Iterative algorithms like K-Means are not inherently optimized for distributed systems like MapReduce!

MapReduce Implementation of K-Means

MapReduce Implementation v1

```
1: class Mapper
2:   method Configure()
3:     c ← LoadClusters()
4:   method Map(id i, point p)
5:     n ← NearestClusterID(clusters c, point p)
6:     p ← ExtendPoint(point p)
7:     Emit(clusterid n, point p)
1: class Reducer
2:   method Reduce(clusterid n, points [p1, p2, ...])
3:     s ← InitPointSum()
4:     for all point p ∈ points do
5:       s ← s + p
6:     m ← ComputeCentroid(point s)
7:     Emit(clusterid n, centroid m)
```

n points
m iterations
d dimensions
k centers

Network traffic!

This MapReduce job performs a single iteration of k-means.

Configure() reads in the current positions of the cluster centers (i.e. stars) from a file.

Map() assigns each point to the closest cluster.

Reduce() aggregates the points in each cluster.

Naïve – Disk I/O between mapper & reducer = O(n m d)

MapReduce Implementation v2 (with in-mapper combiner)

```
1: class Mapper
2:   method Configure()
3:     c ← LoadClusters()
4:     H ← InitAssociativeArray()
5:   method Map(id i, point p)
6:     n ← NearestClusterID(clusters c, point p)
7:     p ← ExtendPoint(point p)
8:     H{n} ← H{n} + p
9:   method Close()
10:    for all clusterid n ∈ H do
11:      Emit(clusterid n, point H{n})
1: class Reducer
2:   method Reduce(clusterid n, points [p1, p2, ...])
3:     s ← InitPointSum()
4:     for all point p ∈ points do
5:       s ← s + p
6:     m ← ComputeCentroid(point s)
7:     Emit(clusterid n, centroid m)
```

Hash Table
Append Point to cluster

This MapReduce job is similar to v1, but uses an in-mapper combiner to combine points in the same map task using a data structure H.

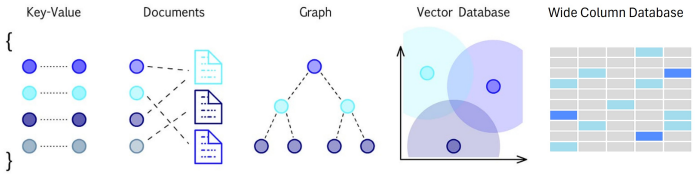
Emit bucket per cluster

- In-mapper combiner used
- High Memory Usage

Optimize – Disk I/O = O(k m d) (# map tasks)*

MapReduce Usage in Data Mining, Machine Learning

- MapReduce still needs to iteratively run till convergence.
- Iterative algorithms like K-Means maybe not ideal for MapReduce, high network, disk I/O overhead (input disk read for every iteration).
- **In-mapper combiner**: aggregate values directly in mapper using in-memory structures to emit fewer k-v pairs, reducing number of combiner calls. Reduce disk I/O since fewer intermediate records, but hgiher memory usage in mapper.
- **System Combiner**: Use dedicated combiner function (run after map phase before data sent to reducer), run by system. More suitable when memory is limited.
- **Secondary Sort** - No aggregation, records (same key) go reducer in specific order.
- While optimizations such as in-mapper combiners can reduce computational cost, but not fully address inefficiencies of iterative algorithms in MapReduce. (Other frameworks e.g. Spark can overcome instead by using caching, in memory)



6. NoSQL & Distributed Databases

NoSQL: non-relational databases, store data in formats other than relational tables.

- "SQL" here refers to traditional relational DBMS (relation databases, not querying language). NoSQL systems often involve SQL-like querying languages.
- "NoSQL" originally "Not SQL," but evolved to mean "Not Only SQL".
- **Coexistence** of relational & relational databases in modern systems, each for tasks they are suited for, e.g. graph, vector databases.
- **Key Motivations**: Growth in web apps, big data, distributed systems.
  - **Volume and Velocity**: Handling large-scale data and high-speed data ingestion.
  - **Scalability**: Horizontal scalability to distribute data across multiple servers.
  - **Flexibility**: Support for unstructured/ semi-structured data w. flexible schemas.
  - **Performance**: Optimize specific use case, e.g. fast read/write, efficient storage.

Major Types of NoSQL Databases

NoSQL databases categorizable into several types, based on their data models:

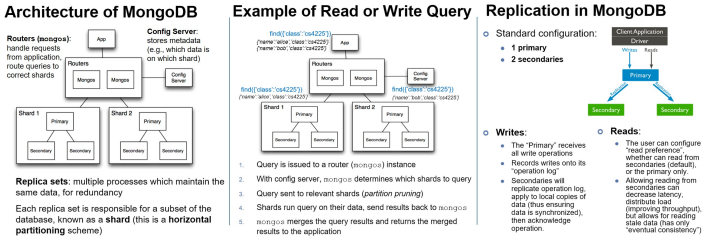
1. **Key-Value Stores**: Store data as key-value pairs (e.g., Redis, DynamoDB).
2. **Wide Column Databases**: Store data in tables with rows and columns, but allow flexible schemas (e.g. Cassandra, HBase).
3. **Document Stores**: Store as documents, JSON-like (e.g. Mongo, CouchDB).
4. **Graph Databases**: Data as nodes, edges to model relationships (e.g. Neo4j).
5. **Vector Databases**: Store high-D vectors for similarity search (e.g., Milvus).

Key-Value Stores

- **Data Model**: Key-value stores associate keys with values.
- **Keys:** Unique identifiers. Primitives e.g. ints, strings, raw bytes, queryable.
- **Values**: can be simple or complex data types, e.g. primitives, lists, JSON objects, or binary large objects (BLOBs). Usually cannot be queried.
- **Operations**: simple API, `Get` (fetch value of key), `Put` (set value associated w. key). Optional operations include Multi-get, multi-put, range queries.
- **Use Cases**: Optimized for fast reads & writes, caching, session storage, small-continuous read/write, uses w. simple query patterns (e.g., Redis, DynamoDB).
- **Persistence**: **Non-persistent** (just big in-memory hashtable, but back up data to disk periodically) e.g. Memcached, Redis. **Persistent**: Data stored persistently to disk. E.g. RocksDB, Dynamo, Riak

Document Stores

- **Data Model**: Organize data into collections of documents.
  - **Database** contains multiple collections. A **collection** contains multiple documents. A **document** is a JSON-like object with fields and values.
  - Documents can be nested (JSON objects as values), varying fields, flexibility for unstructured / semi-structured data.
- **Querying**: Unlike basic k-v stores, document stores support querying based on document content. Common operations: CRUD (create, read, update, delete).
- **Example: MongoDB Architecture**: MongoDB uses a distributed architecture.



Wide Column Databases

Store data in tables w. rows & columns, like relational DB, allow flexible schemas.

- Each row can have different columns, data partitioned by partition key column.
- **Column Stores**: 'Mix between relational database and document store'.
- Ideal for large-scale, distributed datasets (e.g., Cassandra, HBase, open source).

Graph Databases

- Represent data as nodes (entities) and edges (relationships), enabling efficient traversal and querying of relationships.
- These databases are particularly useful for applications like social networks, recommendation engines, and fraud detection, where relationships between entities are critical (e.g., Neo4j).

Vector Databases

- **Vector Databases**: Each row represent point in d-dimensional space.
- Vectors typically represent embeddings from machine learning models. These databases enable fast similarity searches, such as finding nearest neighbors, and are widely used in AI/ML workflows for tasks like recommendation systems, image search, and clustering (e.g., Milvus, Pinecone).
- Algorithms like Locality-Sensitive Hashing (LSH) enable fast similarity search.
- **Examples**: Milvus, Redis, Weaviate, MongoDB Atlas

Pros and Cons of NoSQL Systems

- **Advantages**: Flexible schemas for semi/unstructured data. Horizontally scalable. High performance and availability due to relaxed consistency models.
- **Disadvantages**: Lack of declarative query languages (e.g., SQL). Weaker consistency guarantees, which may require application-level handling. Challenges with schema evolution and data duplication.
- Choice between NoSQL and relational databases depends on: *app requirements* (e.g., consistency vs. availability), query complexity (e.g., joins vs. simple read/writes), data volume and scalability needs.

Key Concepts in NoSQL / Distributed Databases

- **Brewer's / CAP theorem**: Any distributed system/ data store can simultaneously provide only 2 of 3 guarantees: consistency, availability, partition tolerance (CAP).
- **BASE**: Basically Available, Soft state, and Eventually consistent.
  - **ACID** databases prioritize consistency (correctness) over availability—the whole transaction fails if an error occurs in any step within the transaction.
  - **BASE** databases prioritize availability over consistency. Basic read write available most of time. Instead of fail, users access inconsistent data temporarily (*soft state*, no guarantees). Data consistency is achieved eventually.
- **Consistency Models** Different from ACID consistency - data integrity.
  - **Strong Consistency**: Any reads after update give same result on all observers.
  - **Eventual Consistency**: Updates propagate over time, and readers may see stale data temporarily. Offers higher availability but weaker guarantees.
- **Denormalization**: (duplication of data across tables) - tradeoff between query efficiency and propogation of data updates. Design around queries expected.
- **Partitioning** across distributed databases: (e.g. by Range / specific column) - Related queries efficent, (entries co-located, no need scan entire database). However, may have imbalanced partitions, maintenance (shift from one partition to another on change), inefficency of incompatible queries - need scan multiple partitions.

NoSQL Overall: Evaluate pros cons, select right DB system for app use case

- NoSQL DB good for modern apps, flexible dynamic schema, (horizontal) scalability, performance & availability (due to sacrificing strong consistency, removing need for locks, concurrency mechanisms).
- Trade-offs in consistency guarantees (outdated data, additional work to handle on app side), and query capabilities (e.g. no joins).

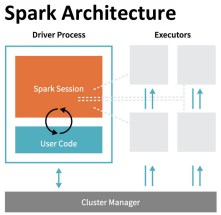
7. Spark

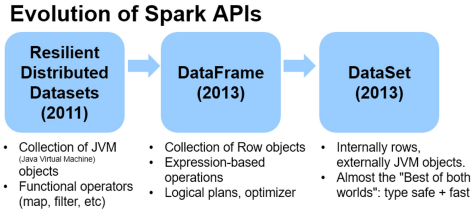
- **Apache Spark**: fast, distributed computing framework, large-scale data processing. Leverage in-memory computation. Outperforms Hadoop MapReduce, especially iterative, interactive workloads, reduced latency, improved throughput.
- **Addresses Limitation**: issue w. Hadoop MapReduce of high network, disk I/O costs as write intermediate data to local disk, shuffle across machines. Not ideal for iterative process (e.g. ML / graph algo), each step a separate MapReduce job.
- **Advantages of Spark**: Store intermediate results in memory, speed up iterative processing. Falls back to disk I/O when memory insufficient.
- **Ease of Programmability**: Simpler, more intuitive API compared to Hadoop MapReduce. E.g. implement WordCount in Spark concise & expressive:

```
val file = sc.textFile("hdfs://...")
val counts = file.flatMap(line => line.split(" "))
                 .map(word => (word, 1))
                 .reduceByKey(_ + _)
counts.save("...")
```

Spark Architecture

- **Driver Process**: Manages Spark application, responds to user input, and distributes work to executors.
- **Executors**: Run the assigned tasks by driver, sends results back to driver.
- **Cluster Manager**: Allocates resources (e.g., Spark's standalone cluster manager, YARN, Mesos, or Kubernetes).
- **Local Mode**: All processes run on a single machine for testing and development.





- Spark achieves simplicity by providing fundamental abstraction of simple logical data structure called Resilient Distributed Dataset (RDD) upon which all other higher-level structured data abstractions, DataFrames and Datasets, constructed.
- Through set of transformations and actions as operations, Spark offers simple programming model to build big data applications in familiar languages.

Resilient Distributed Datasets (RDDs) (Foundational abstraction)

- **RDDs**: A distributed collection of objects that can be processed in parallel.
- **Immutable**: Once created, RDDs cannot be modified.
- **(R) Fault Tolerance**: Achieved through lineage, which allows Spark to recompute lost partitions using the DAG (Directed Acyclic Graph).
- **(DD) Partitioned**: Data divided into partitions distributed across worker nodes.

Transformations and Actions

- **Transformations**: Lazy operations, create new RDDs from existing RDD (e.g., `map`, `filter`, `join`, `groupBy`, `select`). Lazy - nothing happens till action.
- **Actions**: Trigger computations, return results to driver (e.g., `count`, `save`).
- Read, transforms, actions execute in parallel, results sent only in final step.

```
# Create an RDD of names, distributed over 3 partitions
dataRDD = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
nameLen = dataRDD.map(lambda s: len(s)) # Transform
nameLen.collect()  # Action Output: [5, 3, 5, 6]
```

Caching

- Caching: Improve performance by storing frequently used RDDs in memory.
- **When**: Cacje expensive to compute RDD that will be reused multiple times.
- **Methods**: `cache()`: Saves RDD in memory. `persist(options)`: Allows caching in memory, disk, or off-heap memory.
- **Memory Management**: If memory is insufficient, evict LRU RDDs.

Spark Directed Acyclic Graph (DAG)

- Spark constructs DAG to represent RDD transformations and dependencies.
  - **Narrow dependencies**: where each partition of parent RDD used by at most 1 partition of child RDD (e.g. `map`, `contains`)
  - **Wide dependencies**: opposite - each partition of parent RDD used by multiple partitions of child RDD (e.g. `reduceByKey`, `groupBy`)
- **Stages**: In DAG, consecutive narrow dependencies grouped into "stages". Within stages, perform consecutive transforms on same machines.
- **Across Stages**: Wide dependencies trigger *shuffling* (exchange across partitions like mapreduce), which involves write intermediate results to *disk*. Minimize shuffling to improve performance (due to disk I/O not in memory).
- **Fault Tolerance**: Spark uses lineage to recompute lost partitions instead of relying on replication (costly as in-memory). If worker node goes down, replace with new worker, use DAG recompute only RDDs from lost partition.

DataFrames

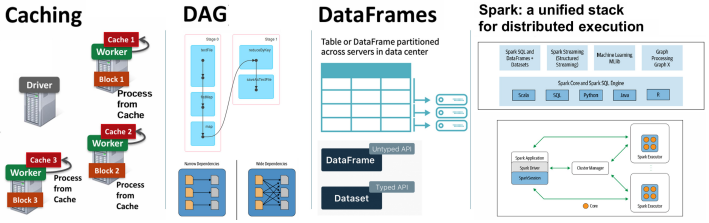
- **Dataframe**: Represent tabular data like SQL tables / pandas DataFrames.
- Provide higher-level interface w. SQL-like operations (e.g. `groupBy`, `orderBy`).

```
flightData2015 = spark.read.option("inferSchema", "true")
                          .option("header", "true")
                          .csv("/path/to/data.csv")
flightData2015.sort("count").take(3)
```

- Generally, transform functions (groupBy, sort) take strings or "column objects".
- Can use SQL queries to transform DataFrames (returns output DataFrame).

Datasets

- DataFrames, but type-safe. (Spark (Scala) - DataFrame alias of Dataset[Row].)
- Available in static typed languages - Scala, but not Python or R. (Dynamic typed)



- RDDs provide low-level control, while DataFrames and Datasets offer ease of use and performance optimizations.
- No matter which abstraction (Df, Ds) used, internally final compute done on RDDs.
- Caching and DAGs are essential for optimizing Spark applications.

Spark Design Philosophy

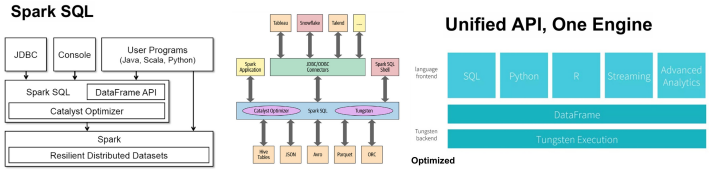
Design philosophy and architecture. Guided by four core principles.

- **Speed**: In-memory processing, optimized execution (Catalyst Opt, Tungsten).
- **Ease of Use**: Unified, intuitive API across languages. High-level abstractions.
- **Modularity**: Modular system, with core enginer supporting libraries and modules. (Spark SQL for structured data, Spark Streaming, MLlib for ML, GraphX). Extensible Ecosystem to build custom libraries.
- **Extensibility**: Support for multiple languages, integration w. existing systems.

Spark SQL

Spark SQL is a module in Spark, provides unified interface for structured and semi-structured data processing. Integrates seamlessly w. Spark's core engine and supports DataFrames and Datasets in Java, Scala, Python, and R.

- **Schema Awareness**: Track schema of data, enable optimized relation operation.
- **Unified API**: Provides a consistent API across multiple programming languages.
- **Optimization**: Leverages the Catalyst Optimizer and Project Tungsten for efficient query execution.



RDD vs DataFrame on Execution Efficiency

- **RDD**: Instructs Spark *how* to compute query, intention opaque to Spark. (no understanding of structure of data, semantics of user-defined functions).
- **DataFrame**: Tells Spark *what* to do, using domain-specific language (DSL) similar to Python pandas. Spark can inspect, optimize query, more efficient execution.

Catalyst Optimizer

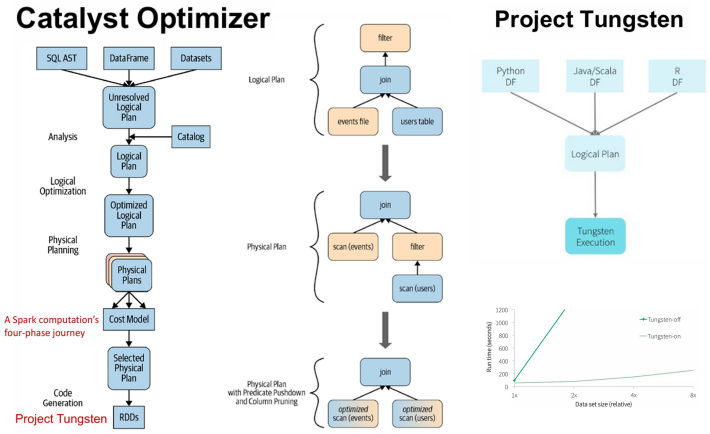
Catalyst Opt. transforms computational queries into execution plans, four phases:

1. **Analysis**: Resolves references and validates the query.
2. **Logical Optimization**: Apply rule-based optimizatn (e.g. pred pushdown).
3. **Physical Planning**: Generates execution plan tailored to underlying hardware.
4. **Code Generation**: Compile optimal plan to low-level code (Proj Tungsten).

Project Tungsten

Proj. Tungsten focuses on improving memory, CPU efficiency by:

- **Memory Management**: Efficiently managing memory usage for large datasets.
- **Binary Processing**: Operating directly on binary data formats.
- **Cache-Aware Computation**: Optimizing for CPU cache utilization.
- **Code Generation**: Generating highly optimized machine code for execution.



Spark ML: Machine Learning

Spark ML provides tools to build scalable, complex machine learning pipelines using simple building blocks. Pipelines streamline the process of data preprocessing, feature engineering, model training, and evaluation. Supports various stages of ML workflow, from data preprocessing to model evaluation.

Typical Machine Learning Pipeline:

1. **Preprocessing**: Cleaning and transforming raw data (e.g., handling missing values, encoding categorical features).
2. **Training**: Fitting a model to the training data.
3. **Testing**: Evaluating the model on unseen data.
4. **Evaluation**: Measure model performance, metrics e.g. precision, recall, F1.

Data Preprocessing

Data preprocessing *crucial for high-quality input* to machine learning models.

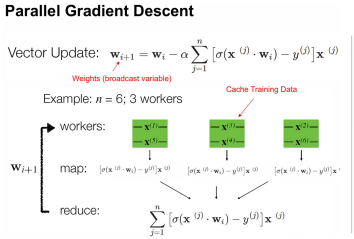
- **Handling Missing Values**: Imputation (e.g., median) / create dummy variables.
- **Categorical Encoding**: Converting categorical features into numerical representations (e.g. one-hot encoding).
- **Normalization**: Scale features to standard range (e.g. min-max norm, log).

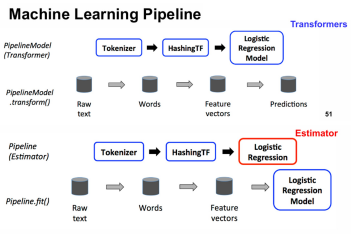
Logistic Regression: widely used classification algorithm. **Key concepts**:

- **Sigmoid Function**: Maps predictions to probabilities between 0 and 1.
- **Cost Function**: Minimizes cross-entropy loss to fit the model parameters.
- **Gradient Descent**: Iterative update model params, minimize cost function.

Parallel Gradient Descent **In distributed systems like Spark**

- Broadcast weights to all worker nodes. Cache training data locally on each worker. (Used to calculate gradients for each iteration). Compute gradients in parallel, aggregate results to update model.





Pipelines in Spark ML

Spark ML pipelines chains multiple stages of data processing and modeling:

- **Why**: Pipelines allow better code reuse, and easier to perform cross validation, hyperparameter tuning (easier to treat as encapsulated).
- **Transformers**: Map DataFrames to DataFrames (e.g. use during one-hot encoding, normalization, model predictions).
- **Estimators**: Algorithm that produces a fitted model (a transformer) (e.g., logistic regression, k-means clustering). `fit()` method that trains model on input data.
- **Pipeline**: Combines transformers and estimators into a single workflow.
- Pipeline chains multiple Transformers and Estimator to form ML workflow.

Pipeline Execution

- **Training Time**: `Pipeline.fit()` called:
  - Transformers execute their `transform()` methods.
  - Estimators execute their `fit()` methods to produce fitted models.
- **Test Time**: Output of `Pipeline.fit()` is a `PipelineModel`, which is a Transformer. Calling `PipelineModel.transform()` applies the transformations and predictions to new data.

8. Stream Processing

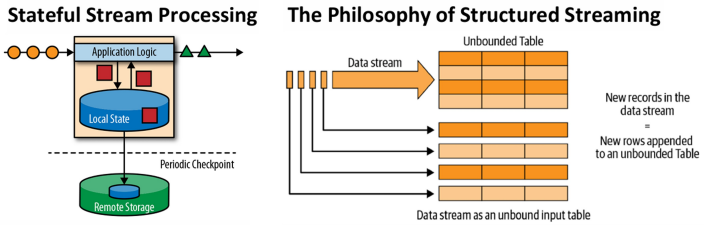
- Scenario where data arrives continuously over time rather than received all at once.
- Streaming / online approaches process input as it is received, compared to offline /batch processing which operate on full dataset at once.

Streaming Data Characteristics

- Input elements (*tuples*) enter rapidly from sources e.g. sensors, TCP connections, file streams, or message queues.
- Streams potentially infinite, cannot be fully stored due to memory limitations.
- Data high volume, constantly arriving over time. Applications:
  - **Sensor Data**: Industrial IoT, environmental monitoring, medical devices.
  - **Online User Activity**: Clickstreams, user interactions, social media feeds.
  - **Financial Transactions**: Credit card fraud detection, stock trades analysis.

Stateful Stream Processing

- **Stream processing systems** not limited to trivial record-at-a-time transformations.
  - Store and access intermediate data. (Local state, periodic checkpoint)
  - Maintain state in prog variables, local files, embedded DBs, external systems.



Spark Structured Streaming

- **Stream processing pipelines** should be as easy as writing batch pipelines.
- Unified programming model and interface for both batch and stream processing.
- Broader definition stream processing: data stream as an unbounded table.

Micro-Batch Stream Processing

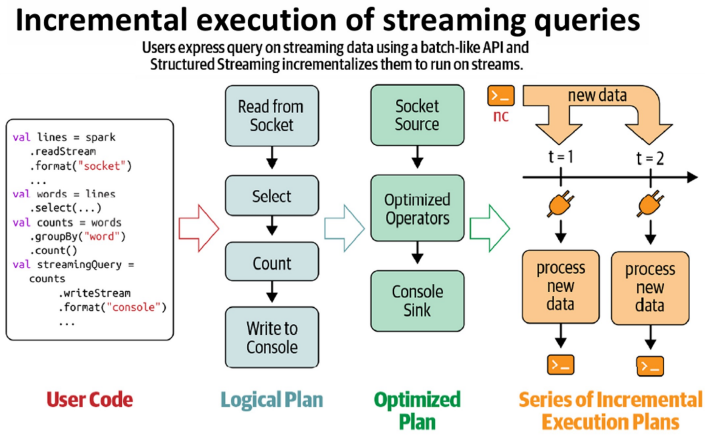
- **Micro-Batch processing model** of Spark Structured Streaming.
  - Divide input stream into small batches.
  - Process each batch in Spark cluster in a distributed manner.
  - Generate output in micro-batches using deterministic tasks.
- **Advantages** of Micro-Batch:
  - Efficient recovery from failures.
  - Ensures end-to-end exactly-once processing guarantees.
- **Disadvantages** of Micro-Batch:
  - Latencies of a few seconds, which may be acceptable for many applications.
  - Applications may incur additional delays in other parts of the pipeline.

Structured Streaming Processing Model

Structured Streaming processes data streams in five steps:

1. **Define Input Sources**: Specify where the data is coming from.
2. **Transform Data**: Apply transformations to the data.
3. **Define Output Sink and Output Mode**: Where, how to write output.
4. **Specify Processing Details**:
   - **Triggering details**: When to trigger results, process new stream data.
   - **Checkpoint location**: Store stream query process information (fail recovery).
5. **Start the Query**: Begin processing the stream.

Incremental execution of streaming queries



Data Transformation (Streaming Data)

Stateless Transformations

- Process each row independently without relying on previous rows. E.g.
- Projection operations: `select()`, `explode()`, `map()`, `flatMap()`.
- Selection operations: `filter()`, `where()`.

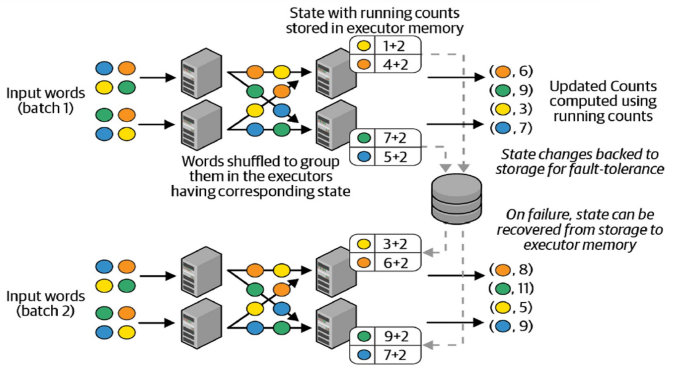
Stateful Transformations

- Maintain intermediate state across records for computations.
- E.g. Aggregations: `DataFrame.groupBy().count()`.
- Every micro-batch, incremental plan adds count of new records to previous count.
- Partial count communicated between plans is the state, maintained in Spark executor memory and checkpointed for fault tolerance.

Distributed State Management

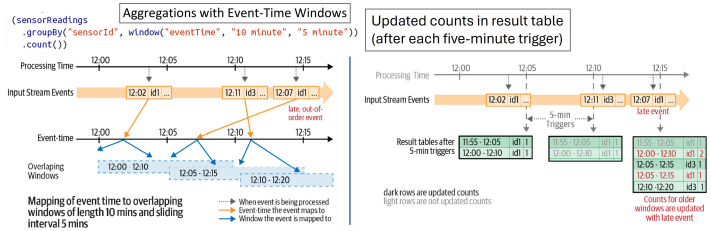
- **Stateful streaming aggregations** include:
  - Global aggregations (E.g. `groupBy().count()`)
  - Grouped aggregations. (E.g. `groupBy('sensorId').mean('value')`
  - Event-Time window Aggregations (E.g. `window('eventTime', '5 min')`)
- All built-in aggregation functions in DataFrames supported, `sum()`, `mean()`, `stddev()`, `countDistinct()`, `collect_set()`, `approx_count_distinct()`.

Distributed state management in Structured Streaming



Time Semantics

- **Processing Time**: Time at which stream processing machine processes event.
- **Event Time**: The time when the event actually occurred.
- **Benefits of Event Time**: Decouples processing speed from results.
  - Operations based on event time are **predictable, deterministic**.
  - Event time window computation yields same result regardless processing speed or event arrival order (out-of-order data arrival).
- **Watermarks**: mechanism handling late-arriving data, state of streaming queries.



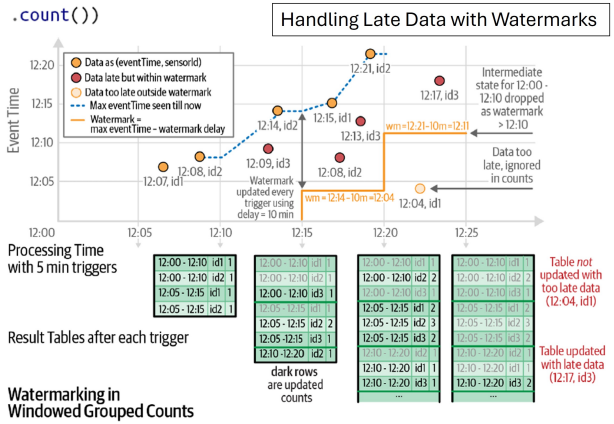
Trigger in Spark Structured Streaming

- **Trigger** define how often and when streaming query process new available data.
- **Default** (Micro-batch Processing): process data ASAP without waiting.
- **Fixed Interval Trigger**: process data at fixed interval (e.g. 5 sec).
- Manage execution of stream query, control latency, throughput, resource usage.

Watermarking in Spark Structured Streaming

- **Defines threshold** for how late data can arrive before considered irrelevant.
  - **Watermark: timestamp**, point in time up to which all data has been processed.
  - Any data arriving w. event time earlier than watermark is "too late", discarded.
- Watermark **dynamically updated** based on maximum observed event time in stream minus a user-specified delay threshold.
- **Watermark Delay Threshold**: user-defined parameter, specify how long Spark waits for late data. E.g. if delay threshold 10 min, Spark assume any data arriving more than 10 minutes after latest observed event time is irrelevant. (Watermark = MaxObservedEventTime − DelayThreshold)

```
(sensorReadings
  .withWatermark("eventTime", "10 minutes")
  .groupBy("sensorId", window("eventTime", "10 minutes", "5 minutes"))
  .count())
```



Performance Tuning

Besides tuning Spark SQL engine, consider following:

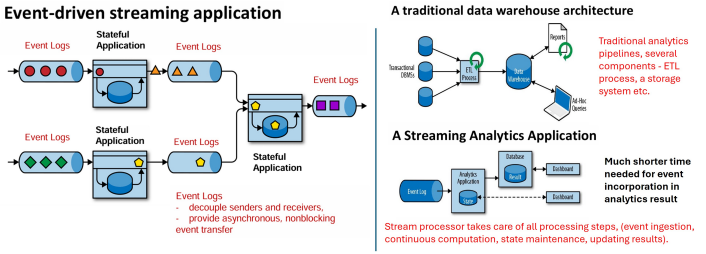
- Appropriate cluster resource provisioning for 24/7 operation.
- Setting number of partitions for shuffles lower than batch queries.
- Limiting source rates for stability.
- Running multiple streaming queries in same Spark application.

Flink

- **Apache Flink**: distributed system for stateful parallel data stream processing.
- Low-latency, high-throughput, fault-tolerant stream processing capabilities.
- Unlike Spark's micro-batch approach, Flink processes events in real-time. (latencies in order of milliseconds).

Event-Driven Streaming Applications

- **Event logs** are ordered, immutable sequences of events (or records) that represent changes or actions in a system. Each event typically includes metadata such as a timestamp, key, and payload (the actual data).
- Logs decouple senders & receivers, enable async, non-blocking event transfer.
- Stateful applications maintain and process state as events are ingested.
- Systems e.g. **Kafka** commonly used as message queues, store, manage event logs.

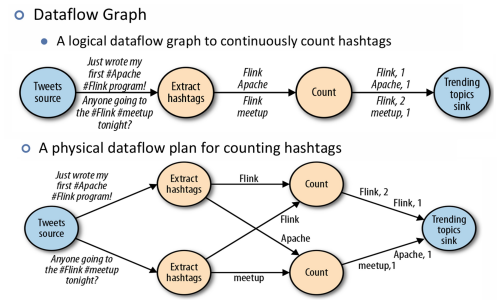


Comparison with Traditional Data Warehousing

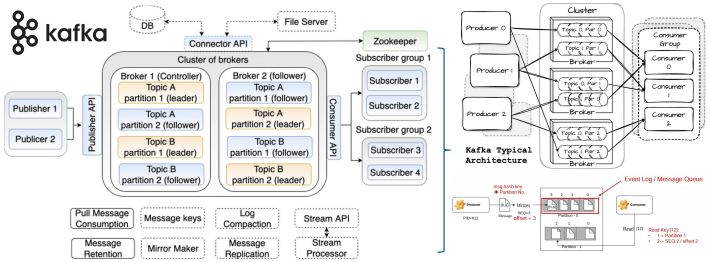
- **Traditional data warehousing architectures**: involve multiple components, ETL (Extract, Transform, Load) pipeline, storage system, batch processing frameworks.
- **Flink's streaming analytics applications**:
  - Handle event ingestion, continuous computation, state maintenance, and result updates within a single framework.
  - Reduce time required for an event to be incorporated into analytics results.

Dataflow Model

- **Logical Dataflow Graph**: operators as nodes and data dependencies as edges.
- **Physical Dataflow Plan**: maps logical operators to tasks for execution. Nodes represents tasks.



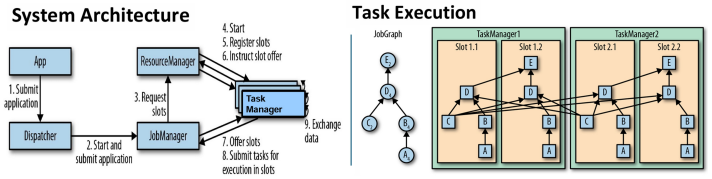
Overview of Apache Kafka



- **Apache Kafka**: distributed event streaming platform for high-throughput, fault-tolerant, real-time data processing. Build real-time data pipelines, streaming apps.
  - **Event Logs**: Kafka stores streams of records (events) in categories called topics.
  - **Decoupling**: Kafka decouples data producers (senders) and consumers (receivers), enabling asynchronous, non-blocking event transfer.
  - **Scalable**: Supports large volumes of data across multiple nodes in a cluster.
  - **Durable**: Data persist on disk, replicate across multiple brokers, fault tolerance.
- **Key Concepts in Kafka**
  - **Topics**: A topic is a category or feed name to which records are published. Topics partitioned, replicated across multiple brokers.
  - **Partitions**: Each topic divided into partitions, allow horizontal scaling. Each partition is ordered, immutable sequence of records.
  - **Producers**: Producers publish data to Kafka topics.
  - **Consumers**: Consumers subscribe to topics and process the data.
  - **Brokers**: Kafka runs as a cluster of brokers. Each broker manages a subset of the partitions for the topics.
  - **Offsets**: Each record in a partition has a unique offset, which acts as an identifier for the record's position in the partition.
- **Partitioning**: partitioning mechanism, distribute data across multiple brokers:
  - **Partition Key**: Messages assigned to partitions based on a key. If no key is provided, messages are distributed in a round-robin fashion.
  - **Hash Function**: Partition number determine using hash function: Partition Number = hash(key)%number of partitions. Ensures all messages with same key sent to the same partition, preserving order within partition.
- **Data Retention**: configurable period or size limit or log compaction (retain only the latest value for each key, reducing storage requirements).
- **Consumer Groups**: allow multiple consumer read from same topic in parallel.
  - **Load Balancing**: Each partition consume by only one consumer within group.
  - **Fault Tolerance**: If consumer fail, partition reassign to other consumer in group.
- **Use Cases of Kafka**: Real-time analytics, event sourcing, microservice comm.
- **Kafka vs Traditional Message Queues**: Data persistence on disk, scalability, order guarantee, replayability.
- **Example: Kafka in Photos Synchronization System**: (e.g. iCloud, GPhotos).
  - **Producer**: Photos uploaded from a phone or laptop are sent to Kafka as events.
  - **Topic**: Events are stored in a Kafka topic, such as "photo-uploads."
  - **Consumer**: Servers subscribe topic, process events, synchronize device photos.
  - **Fault Tolerance**: Photo uploads not lost even if some server fail.
- **Advantages of Kafka**: High Throughput, Low Latency, Durability, Scalability.
- **Challenges with Kafka**: Complexity, Resource Intensive, Latency for small mesages.

Flink System Architecture

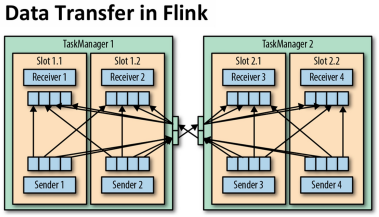
- Flink as distributed system for stateful parallel data stream processing.
- **TaskManager**: Executes tasks and manages data transfer between tasks. A TaskManager can execute multiple tasks concurrently:
  - Tasks of the same operator (data parallelism).
  - Tasks of different operators (task parallelism).
  - Tasks from different applications (job parallelism).
- **JobManager**: Coordinates task execution and initiates checkpoints.

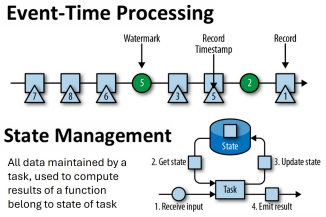


- **Each TaskManager**: fixed number of processing slots to control concurrency.
- **A processing slot** can execute one parallel task slice of an application.

Data Transfer in Flink

- Tasks in a running application continuously exchange data.
- TaskManagers handle data shipping from sending tasks to receiving tasks.
- Records collected in buffers (buffered) before shipping, optimize network utilize.





Flink Event-Time Processing

- **Timestamps**: Every record will include an **event** timestamp (when they occured, vs. using process / ingestion-time timestamps).
- **Watermarks**: Special records indicating progress of event time. Flow alongside regular records, **triggers (event-time-related) computations**. Specified by user!
  - **Periodic Watermarks**: Generated at fixed intervals.
  - **Punctuated Watermarks**: Generate based on specific events in stream.
- **Allowed Lateness**: specifies how much event delay tolerated. Signals: ”All events with timestamps earlier than this watermark have been received.”
- Event arriving after watermark’s timestamp considered ”late”, handled accordingly (e.g., dropped, sent to a side output).
- **Watermarks** implemented as records holding a timestamp (Long value). Tells Flink when to finalize results for given time window. (E.g. watermark at 10:00 indicates Flink can now process all events with timestamps before 10:00).

Flink State Management

- Robust state management for stateful stream processing. **State Backend**:
- **Local State Management**:
  - **Task of a stateful operator** reads, updates its state for each incoming record.
  - **Each parallel task** maintains its state locally in memory for fast access.
- **Checkpointing**:
  - **A TaskManager** process may fail at any point, hence storage considered volatile
  - **Checkpoints persist state** to remote, persistent storage (e.g., distributed filesystems or database system), for fault tolerance to recover from failure.

Checkpoints

Flink’s checkpointing mechanism ensures **consistent** state across distributed tasks: (similar to Spark micro-batch **consistent** checkpoints).

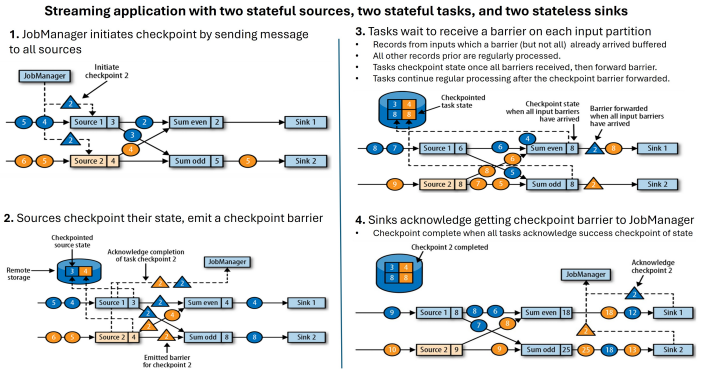
- **Spark (micro-batch) Checkpointing**: Pause ingestion of all input streams, process all in-flight data of all tasks, take checkpoint by copying state of all tasks to remote persistent storage. Resume ingestion of all streams. (‘stop-the-world’)
- **Failure Recovery** (from a consistent checkpoint): Restart entire application, Reset states of all stateful tasks to latest checkpoint. Resume processing from the point of failure. - costly delays.

Flink’s Checkpointing Algorithm

- **Checkpoint algorithm**: based on **Chandy-Lamport** distributed snapshot algo.
- **Checkpoint barriers** inject into stream, cannot overtake or pass by other records.
- Barriers logically split the stream into two parts:
  - State modifications due to records preceding barrier included in checkpoint.
  - Modifications due to subsequent records included in later checkpoint.
- Checkpointing is async, allow tasks to continue process, while others persist state.

Checkpointing Algorithm:

1. JobManager initiates checkpoint, send a message to all source operators.
2. Source operators checkpoint their state and emit a **checkpoint barrier**.
3. Tasks buffer records until **all checkpoint barriers received**.
4. Once all barriers are received, tasks checkpoint state, **forward the barrier**.
5. Sinks acknowledge receipt of the checkpoint barrier to the JobManager.
6. A checkpoint is complete when all tasks have acknowledged their state.



Comparison: Spark vs. Flink

- **Spark**: Micro-batch streaming processing with latencies of a few seconds.
  - Checkpoints are synchronous (”stop the world”).
  - Watermarks are configured to determine when to drop late events.
- **Flink**: Real-time streaming processing with latencies in milliseconds.
  - Checkpoints are asynchronous and distributed, enabling lower latency.
  - Watermarks are special records that trigger event-time-related computations.
  - Late events are handled using watermark-related functions.